

Department of Information Security and Communication Technology

IIK4100 - Hardware Security in Embedded Systems

TP-link - Firmware Modification

Table of Contents

1 Introduction **1**

2 Requirements & Set-up **2**

3 Read the boot log **2**

3.1 Connect to the board 2

3.1.1 Locate the UART 3

3.1.2 Find ground, RX, TX, and Voltage 3

3.1.3 Connect JTAGulator to the UART connection 4

3.2 Communicate with the board 4

3.3 The boot log 8

4 Firmware extraction **8**

4.1 Set-up the miniProgrammer 8

4.2 Locate and connect to the firmware chip 9

4.3 Dump the Firmware 10

4.4 Investigate the operating system 12

4.5 Repack the files 13

5 Software set-up **16**

5.1 Linux 16

5.2 Set up VirtualBox 17

5.3 Give the VM access to CH341A and JTAGulator 17

1 Introduction

This lab will focus on two parts. Part one: Use the UART interface to read the boot process. Part two: Extract the firmware, modify it, and insert the modified firmware back into the router. After both parts are completed, you will be able to see that you have successfully altered the router's firmware. In this lab, we will not engage in any advanced hacking activities but demonstrate the principles of firmware modification and connecting to the UART interface. However, feel free to explore and exploit the router as you wish - the only limitation is your creativity. Before starting the lab, you will be introduced to basic theory on UART and firmware, providing a foundation for understanding what you will do in the lab. If you are already aware of this, you can skip to section 2.

Low-speed Serial Interfaces

One such interface is Universal Asynchronous Receiver/Transmitter (UART), sometimes called RS-232 and TTL Serial. It is mostly used for debugging purposes from the manufacturing side. However, serial interfaces such as UART can result in root access to the system if debugging capabilities are implemented poorly. This is something we want to investigate further when we examine security in embedded systems.

Basics about UART

The UART interface has four connections: Transmit (TX), Receive (RX), Ground (GND) and Voltage (V). In production, where devices are tested through UART, all pins are connected to provide power (Voltage connection), commands (TX connection), response (RX connection), and ground (GND connection). The device uses UART with a specific baud rate, determining the communication speed.

Computer	**Device**
TX - - - - - - -	RX
RX - - - - - - -	TX
GND - - - - - - -	GND
V x x x x x x	V

Table 1: How to connect UART from computer to device.

When connecting a computer to a device to communicate over the UART interface, a specific pattern must be followed to ensure proper operation. The UART interface must be connected as visualized in Table 1. The computer's *TX* needs to connect to the device's *RX* because commands sent from the computer must be received at the device's *RX* interface. The same applies when the device transmits data to the computer. *GND* will remain the same, while *V* will not be connected, as the power supply powers the device in this lab case.

Firmware

Firmware is software embedded in hardware devices to control their low-level functions. It acts as a bridge between the hardware and higher-level software. Firmware can be a target for attacks, as compromising it can grant control over a device, which is why securing the firmware is a crucial part of hardware security in embedded devices.

2 Requirements & Set-up

For this lab, you will need both hardware and software. At the time of writing, only a walkthrough for Linux has been created. You will therefore need a computer with Linux installed, or a Linux virtual machine, to complete this lab. However, you will also need additional tools and software, which will be described in this section. If you have a computer with Windows or Mac OS installed, please download and set up a Kali Linux virtual machine on VirtualBox. Kali Linux is highly recommended since almost all the software packages are already installed in the operating system. You will, however, need to give the virtual machine access to the USB ports. How to set up Kali Linux, the required software and access to the USB ports for the lab, will be available in section 5.

Hardware:	**Software:**
Multimeter	minicom
JTAGulator & USB-B cable	binwalk
miniProgrammer (CH341A)	flashrom
TP-link AC1200 router & power supply	
Jumper cables (male-to-female)	

Table 2: Required hardware and software for the lab.

3 Read the boot log

The first part of the lab involves reading the boot log to visualize the modifications we will make to the firmware later. To do so, we will use the UART debugging port to gain access. Follow these next steps.

3.1 Connect to the board

Before we use the UART port, we can use a multimeter to assume the location of the four different pins on the board. This will make it easier to determine their identity. We will use a JTAGulator to investigate this later.

3.1.1 Locate the UART

First, we need to locate the UART on this router. You can identify it by comparing the image below with the router in front of you.



Figure 1: Location of the UART on the TP-link router.

3.1.2 Find ground, RX, TX, and Voltage

We will use the multimeter to check the four UART connections on the router to determine whether they are ground (GND), receive (RX), transmit (TX), or Voltage (V). Power up the router and measure the four pins. Restart the router between each measurement.

Question 1: What is the voltage measurements on the UART connections?
Write an answer in the report where you enter the measurements from the UART connections. Also, answer which pin you assume is GND, RX, TX and V.

TIP: Locate the ground to the board near the power supply (J4) on the board first. Use this ground to measure the other pins...

We will also use the measurements to set up the JTAGulator before we can read the boot process.

The Figure 2 below visualize how and where it is possible to measure *ground*.



Figure 2: How to located ground with a multimeter.

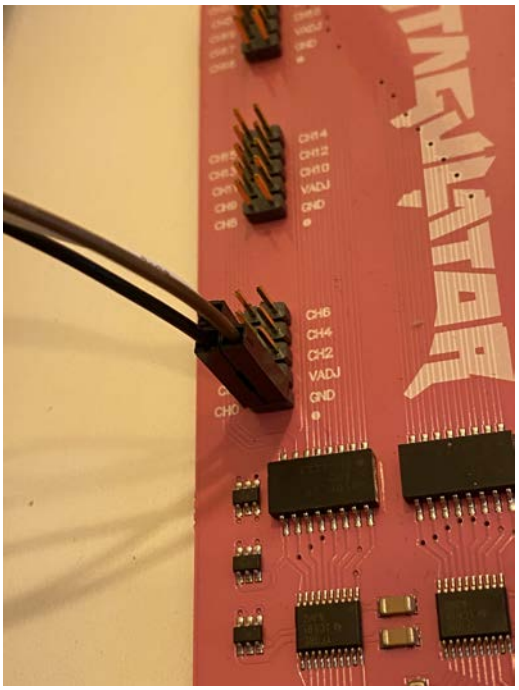


Figure 3: Jumper cables are connected to JTAGulator.

3.1.3 Connect JTAGulator to the UART connection

You will need three jumper cables as described in the *Requirements and Set-up* section above. Connect the three pins to the JTAGulator as shown in Figure 3, where you connect two of the pins to *channel 0* and *channel 1*, and the last one to *GND*. You must connect the *ground* on the router to the GND on the JTAGulator. Connect the Jumper cables to the pins based on the measures and assumptions you made earlier. You do not need to use the *V* pin on the router, only *RX*, *TX* and *GND*.

Connect the JTAGulator to your computer by USB, and give the VM access to the USB port where the JTAGulator is connected (details on how to give access is provided in Section 5). Open a terminal on your VM and run the following command in the Linux environment:

```
sudo minicom -D /dev/ttyUSB0 -b 115200
```

Once you are in a terminal, hit enter one time, and you should be prompted with the *JTAGulator* logo as shown in Listening 1.

3.2 Communicate with the board

As you are ready to start JTAGulating, follow the terminal input below to identify the pins and find the correct *baud rate*. First, the Voltage needs to be set to 3.3, as we measured earlier with

the multimeter, and we chose “T” to find this device’s *baud rate*. After finding the *baud rate*, we can use the command “P” to read the terminal. However, we still need to figure out which of the two pins that are *TX* and *RX*, so we will have to test the UART passthrough two times at most to figure out which pins are *TX* and *RX*.

```
ubuntu@XPS:~$ sudo minicom -D /dev/ttyUSB0 -b 115200
[sudo] password for ubuntu:

Welcome to minicom 2.9

OPTIONS: I18n
Port /dev/ttyUSB0, 20:47:37

Press CTRL-A Z for help on special keys




                                    UU  LLL
 JJJ  TTTTTTT AAAAA  GGGGGGGGGG   UUUU LLL   AAAAA TTTTTTTT OOOOOOO  RRRRRRRRR
 JJJJ TTTTTTT AAAAAA GGGGGGG      UUUU LLL  AAAAAA TTTTTTTT OOOOOOO  RRRRRRRR
 JJJJ   TTTT  AAAAAAA GGG      UUU  UUUU LLL  AAA AAA    TTT  OOOO OOO  RRR RRR
 JJJJ   TTTT  AAA AAA GGG  GGG UUUU UUUU LLL AAA  AAA    TTT  OOO  OOO  RRRRRRR
 JJJJ   TTTT  AAA  AA GGGGGGGGG UUUUUUUU LLLLLLLL AAAA   TTT OOOOOOOOO  RRR RRR
  JJJ   TTTT AAA   AA GGGGGGGGG UUUUUUUU LLLLLLLLL AAA   TTT OOOOOOOOO  RRR RRR
  JJJ   TT                  GGG             AAA                        RR RRR
 JJJ                         GG             AA                             RRR
JJJ                           G             A                                RR


           Welcome to JTAGulator. Press 'H' for available commands.
         Warning: Use of this tool may affect target system behavior!

> H
Target Interfaces:
J   JTAG
U   UART
G   GPIO
S   SWD

General Commands:
V   Set target I/O voltage
I   Display version information
H   Display available commands

> V
Current target I/O voltage: Undefined
Enter new target I/O voltage (1.4 - 3.3, 0 for off): 3.3
New target I/O voltage set: 3.3
Warning: Ensure VADJ is NOT connected to target!

> U
```

```
47
48 UART> H
49 UART Commands:
50 U    Identify UART pinout
51 T    Identify UART pinout (TXD only, continuous)
52 P    UART passthrough
53
54 General Commands:
55 V    Set target I/O voltage
56 H    Display available commands
57 M    Return to main menu
58
59 UART> T
60 UART pin naming is from the target's perspective.
61 Enter starting channel [0]: 0
62 Enter ending channel [0]: 1
63
64 Ignore non-standard baud rates? [y/N]:
65 Press spacebar to begin (any other key to abort)...
66 JTAGulating! Press any key to abort...
67 ----
68 TXD: 0
69 Baud (Measured): 1904761
70 Baud (Best Fit): N/A
71 -
72 TXD: 0
73 Baud (Measured): 98400
74 Baud (Best Fit): N/A
75
76 TXD: 0
77 Baud (Measured): 769230
78 Baud (Best Fit): N/A
```

Listing 1: How to find the baud rate with JTAGulator

Question 2: What is the baud rate of the UART interface on this router?

Write an answer in the report where you enter the baud rate measured from the ***"T Identify UART pinout (TXD only, continuous)"*** *option in JTAGulator as visualized above. Also, provide a screenshot of the terminal.*

As we have now figured out the baud rate, we must investigate which pins are *RX* and *TX*. Follow the terminal output provided below:

NOTE: on the lines where there are no characters, the key "Enter" is pressed to confirm the settings.

Testing *channel 0* as TX and *channel 1* as RX:

```
UART> P
UART pin naming is from the target's perspective.
Enter X to disable either pin, if desired.
Enter TXD pin [0]: 0
Enter RXD pin [0]: 1
Enter baud rate [0]: fill in your baud rate here
Enable local echo? [y/N]:
Entering UART passthrough! Press Ctrl-X to exit...
" % ??    ? ``` *    * @    2 +
```

Listing 2: Finding TX (channel 0) and RX (channel 1)

Testing *channel 1* as TX and *channel 0* as RX:

```
UART> P
UART pin naming is from the target's perspective.
Enter X to disable either pin, if desired.
Enter TXD pin [0]: 1
Enter RXD pin [1]: 0
Enter baud rate [0]: fill in your baud rate here
Enable local echo? [y/N]:
Entering UART passthrough! Press Ctrl-X to exit...
U-Boot 1.1.3 (Aug 10 2023 - 10:26:37)

Board: Ralink APSoC DRAM:  64 MB
relocate_code Pointer at: 83fb8000
gpiomode1 54154404.
gpiomode2 00000555.
flash manufacture id: 20, device id 40 17
Warning: un-recognized chip ID, please update bootloader!
============================================
Ralink UBoot Version: 4.3.0.0
--------------------------------------------
ASIC 7628_MP (Port5<->None)
DRAM component: 512 Mbits DDR, width 16
DRAM bus: 16 bit
...
```

Listing 3: Finding TX (channel 1) and RX (channel 0)

We have now successfully found all the pins to the UART. The image below presents where the *TX*, *RX*, *GND*, and *V* are located.

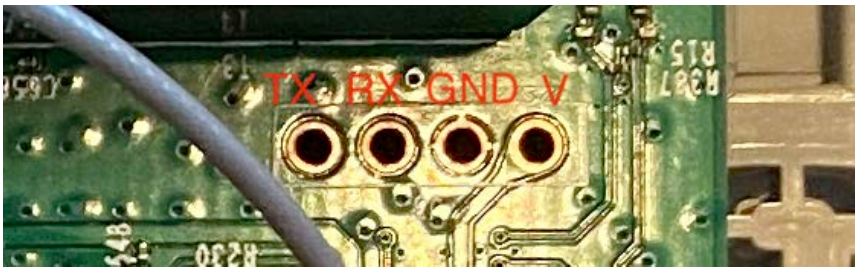


Figure 4: UART located on the router: TX, RX, GND, V

3.3 The boot log

Restart the router and watch the boot log. You might have to pull up the cable located at the TX on the router. Sometimes the router refuses to start up if the pin is connected to TX. So, restart the router, and immediately after you see the LED lights blink, you can reconnect the cable to the TX location.

Question 3: What is the Public RSA key printed in your boot log?

Provide a screenshot of the public RSA key in the answer.

4 Firmware extraction

The next part of the lab will focus on extracting and modifying the firmware. If you managed to get the UART working, you will probably see some familiar lines from the boot log in the file system we will extract.

4.1 Set-up the miniProgrammer

Before connecting the mini programmer to the board or PC, ensure that the MiniProgrammer is assembled as shown in Figure 5.

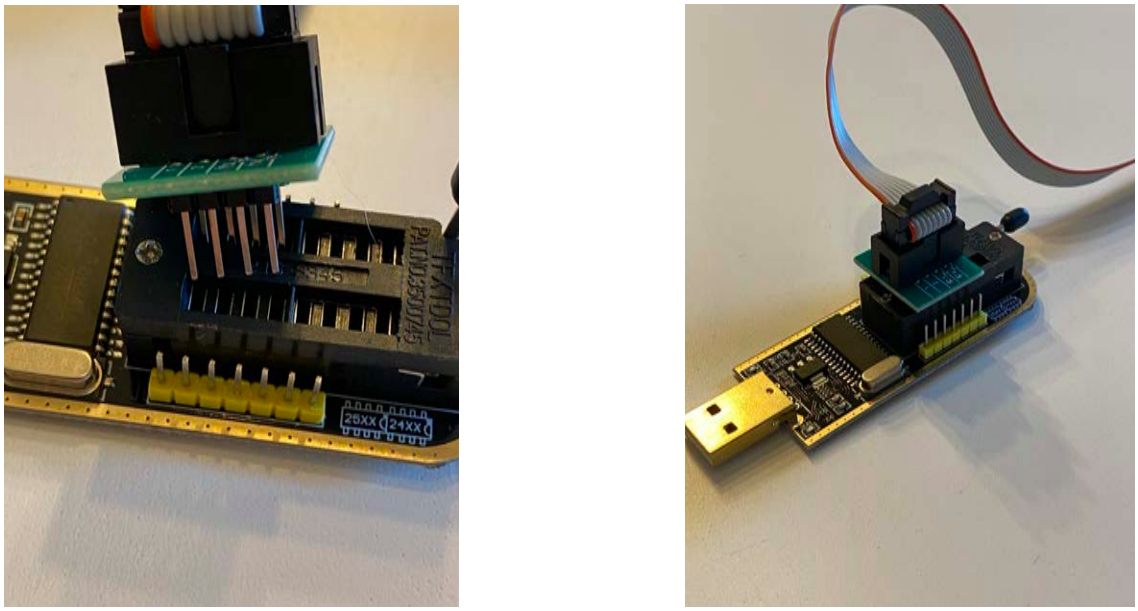


Figure 5: Make sure you place the pins exactly as visualized above, and that the red wire is facing the USB port.

4.2 Locate and connect to the firmware chip

Before connecting to the router, ensure it is powered off. The image to the left is the location of the chip, and the image to the right is how you connect to the chip. Also, ensure that the red wire faces the LED lights on the front.

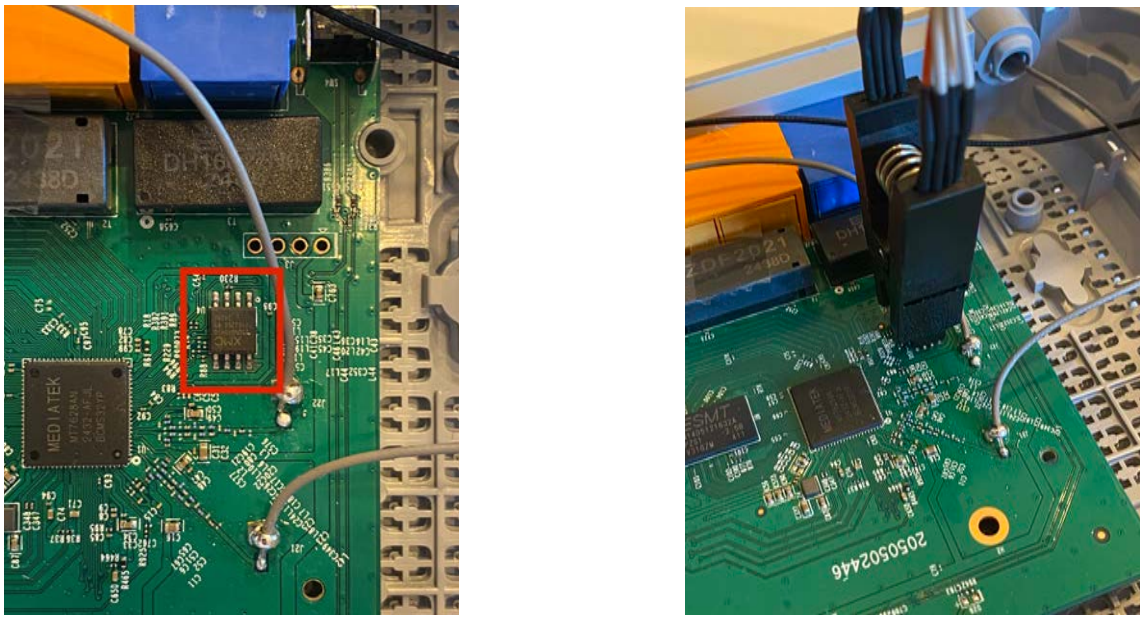


Figure 6: Visualization of how the clip is supposed to be connected to the chip.

4.3 Dump the Firmware

To dump the firmware, you need to position the clip on the chip and run the following command to read and dump the firmware in one go. The command uses the miniProgrammer *ch341a* to *read* the contents of the chip and store it to a file named *firm.bin*.

```
flashrom -p ch341a_spi -r firm.bin
```

Expected output:

```
ubuntu@XPS:~$ flashrom -p ch341a_spi -r firm.bin
flashrom unknown on Linux 6.11.0-25-generic (x86_64)
flashrom is free software, get the source code at https://flashrom.org/

Using clock_gettime for delay loops (clk_id: 1, resolution: 1ns).
Found XMC flash chip "XM25QH64C" (8192 kB, SPI) on ch341a_spi.
===
This flash part has status UNTESTED for operations: PROBE READ ERASE WRITE WP
The test status of this chip may have been updated in the latest development
version of flashrom. If you are running the latest development version,
please email a report to flashrom@flashrom.org if any of the above operations
work correctly for you with this flash chip. Please include the flashrom log
file for all operations you tested (see the man page for details), and mention
which mainboard or programmer you tested in the subject line.
Thanks for your help!
Reading flash... done.
```

Listing 4: Desirable result after running *flashrom -p ch341a_spi -r firm.bin*

Unfortunately, the clip can be somewhat deviant. And it can sometimes be hard to make it work properly.

If the miniprogrammer sends the following message, you must reposition the clip on the chip until it can read it.

Unexpected output:

```
ubuntu@XPS:~$ flashrom -p ch341a_spi -r firm.bin
flashrom unknown on Linux 6.11.0-25-generic (x86_64)
flashrom is free software, get the source code at https://flashrom.org

Using clock_gettime for delay loops (clk_id: 1, resolution: 1ns).
No EEPROM/flash device found.
Note: flashrom can never write if the flash chip isn't found automatically.
```

Listing 5: Flashrom error message after running *flashrom -p ch341a_spi -r firm.bin*

When you can read the firmware, do it again to ensure you have extracted it. We have a copy of the firmware on Blackboard if you lose it or brick the router.

After flashrom has dumped the firmware, run *binwalk* to inspect the contents of the firmware:

```
binwalk firm.bin
```

Depending on your Linux distribution, you will get variations when running the command of the *binwalk*. You should receive something like this output:

Expected output:

```
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
96784         0x17A10         U-Boot version string, "U-Boot 1.1.3 (Aug 10 2023 -
    10:26:39)"
112079        0x1B5CF         HTML document header
131674        0x2025A         HTML document footer
277840        0x43D50         U-Boot version string, "U-Boot 1.1.3 (Aug 10 2023 -
    10:26:37)"
328192        0x50200         LZMA compressed data, properties: 0x5D, dictionary
    size: 8388608 bytes, uncompressed size: 3441784 bytes
1638400       0x190000        Squashfs filesystem, little endian, version 4.0,
    compression:xz, size: 6331236 bytes, 742 inodes, blocksize: 131072 bytes,
    created: 2025-05-30 13:29:41
```

Listing 6: Example result after running *binwalk firm.bin* on Ubuntu

If the output of *binwalk firm.bin* is empty or consists of many *xz compressed* fields like this example below, you will need to dump the firmware again with the *flashrom -p ch341a_spi -r firm.bin* command.

Unexpected output:

```
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
96784         0x17A10         U-Boot version string, "U-Boot 1.1.3 (Aug 10 2023 -
    10:26:39)"
112079        0x1B5CF         HTML document header
131674        0x2025A         HTML document footer
277840        0x43D50         U-Boot version string, "U-Boot 1.1.3 (Aug 10 2023 -
    10:26:37)"
328192        0x50200         LZMA compressed data, properties: 0x5D, dictionary
    size: 8388608 bytes, uncompressed size: -1 bytes
1638400       0x190000        Squashfs filesystem, little endian, version 4.0,
    compression:xz, size: 6331236 bytes, 742 inodes, blocksize: 131072 bytes,
    created: 2025-05-30 13:29:41
7979776       0x79C300        xz compressed data
7999100       0x7A0E7C        xz compressed data
8016052       0x7A50B4        xz compressed data
8051640       0x7ADBB8        xz compressed data
...
```

Listing 7: Read error after running *flashrom -p ch341a_spi -r firm.bin*

After you successfully dumped the firmware, make sure you copy it so you have a backup. Use this command to create a copy:

```
cp firm.bin mod_firm.bin
```

After creating a copy, use the following command to extract the *Squashfs filesystem (squashfs.img)* from the firmware. The file we will extract is the router's operating system, which we would like to investigate further. The following command extracts the contents of *DECIMAL:* 1638400 (*squashfs filesystem*) from the firmware, and the contents will be stored in the file *squashfs.img*.

```
dd if=mod_firm.bin of=squashfs.img bs=1 skip=1638400
```

Run the next command to make the content of *squashfs.img* readable and possible to navigate.

```
sudo unsquashfs -d squashfs-root squashfs.img
```

You should now have a folder named *squashfs-root* with content to explore.

4.4 Investigate the operating system

Optional question: What is the password to the *admin* user?

Hint: etc/ folder

Modify the rcS startup script

The central part of the exercise is to modify the firmware, but what you change is up to you. As a demo of the concept, we will add a basic *echo* in the startup script to visualize that we managed to modify the firmware.

Locate the rcS file

If you are standing in the *squashfs-root* folder, this should be the path to rcS: **etc/init.d/rcS**

Open a text editor of your choice and scroll to the bottom of the file, place a *"sleep 60"* and the *echo* command under the *cos &* line. The *"sleep 60"* is added so the echo is stalled to the end of the boot, which makes it easier to locate.

```
1 cos &
2
3 sleep 60
4 echo '-Firmware_modification_is_pretty_easy-'
```

Listing 8: Example of firmware modification

Save the file and navigate out of the *squashfs-root* folder.

4.5 Repack the files

We will now repack the modified *squashfs-root* folder to *squashfs.img*, and further add *squashfs.img* into the *mod_firm.bin* file.

Note: We are now writing the modified part of squashfs-root into the copy of the firmware we created earlier (mod_firm.bin).

Repack the *squashfs.img* file with the following command:

```
mksquashfs squashfs-root/ squashfs.img -comp xz -b 131072 -noappend
```

Verify that the contents of *squashfs.img* are not larger than **7Mb** (6.1 to 6.5 Mb is normal) with the following command:

```
ls -lh squashfs.img
```

After you have verified the size, you can use the next command to add *squashfs.img* to the firmware (mod_firm.mod):

```
dd if=squashfs.img of=mod_firm.bin bs=1 seek=1638400 conv=notrunc
```

Verify that the modified firmware *mod_firm.bin* has the same addresses and names as the first time you ran the binwalk command. Run *binwalk mod_firm.bin* again to verify.

Expected result:

```
1 DECIMAL       HEXADECIMAL     DESCRIPTION
2 --------------------------------------------------------------------------------
3 96784         0x17A10         U-Boot version string, "U-Boot 1.1.3 (Aug 10 2023 -
      10:26:39)"
4 112079        0x1B5CF         HTML document header
5 131674        0x2025A         HTML document footer
6 277840        0x43D50         U-Boot version string, "U-Boot 1.1.3 (Aug 10 2023 -
      10:26:37)"
7 328192        0x50200         LZMA compressed data, properties: 0x5D, dictionary
      size: 8388608 bytes, uncompressed size: 3441784 bytes
8 1638400       0x190000        Squashfs filesystem, little endian, version 4.0,
      compression:xz, size: 6331272 bytes, 742 inodes, blocksize: 131072 bytes,
      created: 2025-06-02 17:35:54
```

Listing 9: Example of firmware modification. On the entry at line 8 we can see that the date is correct.

The next step is to *write* the modified firmware back to the chip on the router (use the mini-Programmer *ch341a*). After you have placed the clip onto the firmware chip, use the following command to write the modified firmware to the chip:

```
flashrom -p ch341a_spi -w mod_firm.bin
```

Expected result:

```
flashrom unknown on Linux 6.11.0-25-generic (x86_64)
flashrom is free software, get the source code at https://flashrom.org/

Using clock_gettime for delay loops (clk_id: 1, resolution: 1ns).
Found XMC flash chip "XM25QH64C" (8192 kB, SPI) on ch341a_spi.
===
This flash part has status UNTESTED for operations: PROBE READ ERASE WRITE WP
The test status of this chip may have been updated in the latest development
version of flashrom. If you are running the latest development version,
please email a report to flashrom@flashrom.org if any of the above operations
work correctly for you with this flash chip. Please include the flashrom log
file for all operations you tested (see the man page for details), and mention
which mainboard or programmer you tested in the subject line.
Thanks for your help!
Reading old flash chip contents... done.
Erasing and writing flash chip... Erase/write done.
Verifying flash... VERIFIED.
```

Listing 10: Desirable result after running *flashrom -p ch341a_spi -w mod_firm.bin*

Remove the clip from the board, connect the power supply to the router and try to start the router. If you managed to brick the router, you can try to write the modified firmware back to the router again. If that does not work, you will have to create a new copy of *firm.bin* and start from there again.

Otherwise, if you managed to write the firmware back to the chip and the router works, you should connect to the UART port and watch if your *echo* appears near the *RSA* and *dss* keys that are being generated near the end of the boot process.

Result after connecting to the UART port:

```
...
Generating 1024 bit dss key, this may take a while...
[ util_execSystem ] 185:  oal_sys_disableInternetLed cmd is "echo "0 0" > /proc/
    tplink/led_internet"

[ util_execSystem ] 185:  oal_sys_disableInternetLed cmd is "echo "0 0" > /proc/
    tplink/internet_status"

- Firmware modification is pretty easy -
starting pid 1270, tty '': '/bin/sh'
starting pid 1272, tty '/dev/console': '/bin/sh'
/bin/sh: can't access tty; job control turned off
~ # /bin/sh: can't access tty; job control turned off
/ # Public key portion is:
...
```

Listing 11: Desirable result after modifying the firmware

Line 7 visualizes the echo we placed into the firmware. We have now modified the firmware and demonstrated how to do it with relatively cheap tools. Feel free to further explore the router's firmware; it is acceptable to search for exploits and make modifications as you desire.

Question 4: Provide a screenshot of the boot log, where you successfully modified the firmware.

Answer the question with a screenshot of your terminal.

Question 5: How can the UART interface and firmware be secured?

The answer should contain some examples.

— End of Lab —

5 Software set-up

To complete this lab, you will need several programs. Depending on your operating system, you may have to download all the software specified in this section. If you are a Mac or Windows user, please set up a Kali Linux virtual machine (VM). Linux users only need to download the specific software we will use.

5.1 Linux

Minicom

1. Open a terminal and run the following command:

   ```
   sudo apt install minicom
   ```

2. After installation, run this command to confirm the install:

   ```
   minicom --version
   ```

3. Congratulations, you have now installed *minicom* on your computer/vm.

Binwalk

1. Open a terminal and run the following command:

   ```
   sudo apt install binwalk
   ```

2. After installation, run this command to confirm the install:

   ```
   binwalk --version
   ```

   or

   ```
   binwalk -h
   ```

3. Congratulations, you have now installed *binwalk* on your computer/vm.

Flashrom

1. Open a terminal and run the following command:

   ```
   sudo apt install flashrom
   ```

2. After installation, run this command to confirm the install:

   ```
   flashrom --version
   ```

or

```
flashrom -h
```

3. Congratulations, you have now installed *flashrom* on your computer/vm.

5.2 Set up VirtualBox

1. Download and VirtualBox at their homepage: https://www.virtualbox.org/wiki/Downloads
2. After downloading the software, run through the installation and setup.
3. Download the latest version of Kali Linux from: https://www.kali.org/get-kali/#kali-installer-images
4. To set up a Kali Linux VM, you need to do the following:
   (a) When you have opened VirtualBox, click on the blue icon with the description *"New"* to start creating a VM.
   (b) Give the VM a name, choose the Kali Linux ISO you downloaded, and click *"Next"*
   (c) Give the VM at least *8GB Base Memory*, half of your CPUs, and 25 GB Disk Size is Ok.
   (d) On the Summary, click "Finish" to start the configuration.
   (e) You should now see that VirtualBox is setting up the VM, but you will have to interact with the process. One recommendation is to use the GUI install if you are unfamiliar with the command line.
   (f) The rest of the install should be straightforward. If you encounter any problems with the install, please get in touch with the TAs.
5. Also, remember to update and upgrade the OS after installation. Run *sudo apt update && upgrade*.
6. Ensure that Kali VM has the tools described in Section 5.1.

5.3 Give the VM access to CH341A and JTAGulator

To let the VM have access to the USB port for the miniprogrammer (CH341A), you need to connect the miniprogrammer to your computer, and do the following after successfully installing VirtualBox and you have a running instance of Kali Linux:

1. Open VirtualBox → Select your Kali VM → Settings → Select USB → Enable USB controller → Select USB 2.0 or USB 3.0
2. Launch your Kali VM → open a terminal at the desktop.

3. Run *lsusb* to list all USB connections to your Kali. You should see something like this:

```
└─$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 80ee:0021 VirtualBox USB Tablet
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
```

Figure 7: Result after running *lsusb*.

4. Navigate to the bottom right corner → right-click on the USB icon:



Figure 8: USB icon on the toolbar.

5. Choose the USB device named *QinHeng Electronics CH341 ...*:

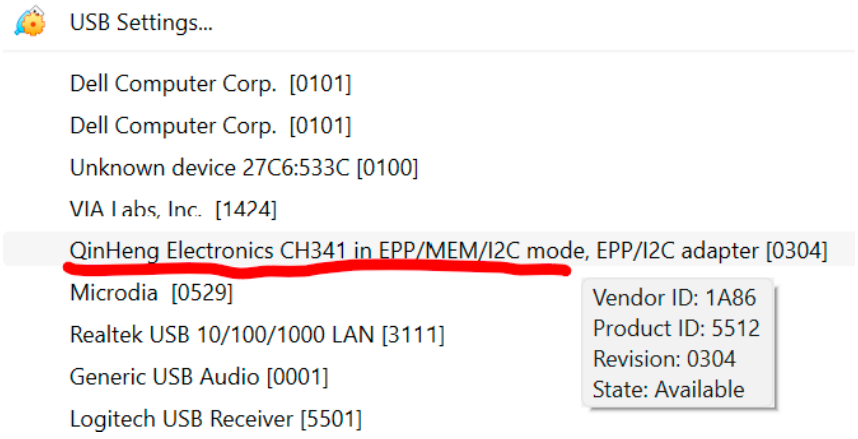


Figure 9: Example output of USB devices.

6. Run *lsusb* to list all USB connections again:

```
└─$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 80ee:0021 VirtualBox USB Tablet
Bus 001 Device 003: ID 1a86:5512 QinHeng Electronics CH341 in EPP/MEM/I2C mode, EPP/I2C adapter
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
```

Figure 10: the miniProgrammer CH341A is now connected to the VM.

7. Do the same thing to add the JTAGulator to the VM. Right-click on the USB icon → choose the USB device named *FTDI FT232R USB UART*:

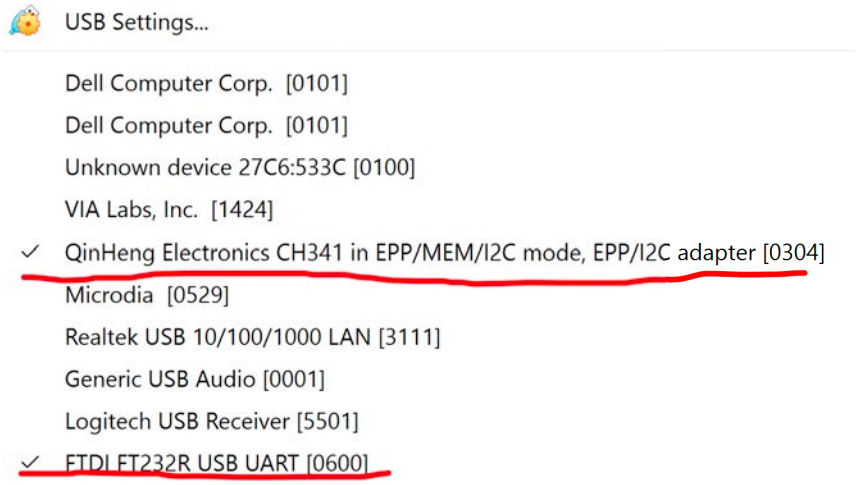


Figure 11: JTAGulator and CH341A are now selected at the same time.

8. Run *lsusb* to see that JTAGulator and CH341A (miniProgrammer) is connected to the VM:

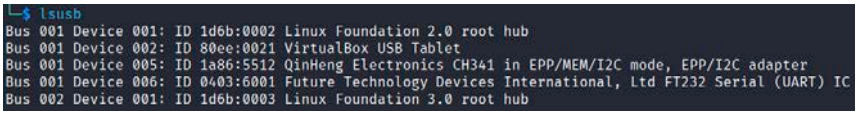


Figure 12: JTAGulator and CH341A (miniProgrammer) are connected.

You have now successfully given the VM access to the CH341A and JTAGulator.